

Recursos Intermediários do Java

Guia Completo para Iniciantes

Conhecendo Recursos Usados no Dia a Dia

Nas fases anteriores você aprendeu a base do Java: sintaxe, lógica e Orientação a Objetos. Agora chegou a hora de conhecer os recursos que os desenvolvedores Java usam no dia a dia do mercado.

O que você vai aprender nesta fase:

- Collections Framework — a forma profissional de trabalhar com listas, conjuntos e mapas
- Tratamento de Exceções — como lidar com erros de forma elegante e segura

Esses dois tópicos aparecem em praticamente todo projeto Java real!

Collections Framework

Lembra dos arrays? Eles funcionam, mas têm um problema: o tamanho é fixo. O Collections Framework resolve isso com estruturas de dados dinâmicas — elas crescem e encolhem conforme a necessidade.

Visão Geral das Collections

Interface	Implementações	Característica principal
List	ArrayList, LinkedList	Ordenada, permite duplicatas, acesso por índice
Set	HashSet, TreeSet	Sem duplicatas, sem índice
Map	HashMap, TreeMap	Chave → Valor, chaves únicas

6.1 List — Listas Ordenadas

Uma List é uma coleção ordenada que permite elementos duplicados. Você acessa os elementos por índice, exatamente como em um array — mas com tamanho dinâmico.

ArrayList vs LinkedList

ArrayList

Armazena elementos em um array interno que se expande automaticamente.

LinkedList

Cada elemento aponta para o próximo (lista encadeada).

- Acesso rápido por índice (get/set)
- Ideal para leitura frequente
- Mais lento para inserir/remover no meio
- Uso mais comum no dia a dia

- Inserção/remoção rápida no início e fim
- Acesso por índice mais lento
- Ideal para filas e pilhas
- Consome mais memória

ArrayList na prática:

```
import java.util.ArrayList;
import java.util.List;

public class ExemploList {
    public static void main(String[] args) {
        // Criando um ArrayList
        List<String> nomes = new ArrayList<>();

        // Adicionando elementos
        nomes.add("Ana");
        nomes.add("Bruno");
        nomes.add("Carlos");
        nomes.add("Ana"); // duplicata permitida!

        // Acessando por índice
        System.out.println("Primeiro: " + nomes.get(0)); // Ana
        System.out.println("Tamanho: " + nomes.size()); // 4

        // Percorrendo com for-each
        for (String nome : nomes) {
            System.out.println("- " + nome);
        }

        // Verificando existência
        System.out.println(nomes.contains("Bruno")); // true

        // Removendo
        nomes.remove("Carlos");
        nomes.remove(0); // remove pelo índice

        System.out.println("Após remoções: " + nomes);
    }
}
```

Métodos essenciais do List:

Método	O que faz	Exemplo
add(elemento)	Adiciona ao final	lista.add("João")
add(índice, elem)	Inserir na posição	lista.add(0, "Ana")
get(índice)	Retorna o elemento	lista.get(2)
set(índice, elem)	Substitui elemento	lista.set(1, "novo")
remove(índice)	Remove por posição	lista.remove(0)
remove(objeto)	Remove por valor	lista.remove("Ana")

<code>size()</code>	Quantidade de elementos	<code>lista.size()</code>
<code>contains(elem)</code>	Verifica se existe	<code>lista.contains("X")</code>
<code>clear()</code>	Remove todos	<code>lista.clear()</code>
<code>Collections.sort(lista)</code>	Ordena a lista	<code>Collections.sort(nomes)</code>

6.2 Set — Conjuntos Sem Duplicatas

Um Set é uma coleção que NÃO permite elementos duplicados. Pense nele como um cadastro onde cada item deve ser único — como CPFs ou e-mails.

HashSet

Armazena sem ordem definida. O mais rápido para verificar se um elemento existe.

- Sem ordem de inserção
- `add()`, `remove()`, `contains()` = $O(1)$
- Permite null
- Melhor para verificações rápidas

TreeSet

Mantém os elementos em ordem natural (crescente).

- Ordenado automaticamente
- `first()`, `last()`, `headSet()`, `tailSet()`
- Não permite null
- Ideal para dados ordenados

HashSet e TreeSet na prática:

```
import java.util.HashSet;
import java.util.TreeSet;
import java.util.Set;

public class ExemploSet {
    public static void main(String[] args) {
        // HashSet - sem ordem
        Set<String> emails = new HashSet<>();
        emails.add("ana@email.com");
        emails.add("bruno@email.com");
        emails.add("ana@email.com"); // IGNORADO! Já existe

        System.out.println("Emails únicos: " + emails.size()); // 2
        System.out.println(emails.contains("ana@email.com")); // true

        // TreeSet - com ordem alfabética
        Set<String> cidades = new TreeSet<>();
        cidades.add("São Paulo");
        cidades.add("Aracaju");
        cidades.add("Manaus");
        cidades.add("Belém");

        // Exibe em ordem alfabética:
        for (String c : cidades) {
            System.out.println(c); // Aracaju, Belém, Manaus, São Paulo
        }
    }
}
```

```
}  
}
```

6.3 Map — Dicionários Chave → Valor

Um Map associa uma chave a um valor, como um dicionário. Cada chave é única, mas os valores podem se repetir. Pense em CPF → Nome, Código → Produto, Palavra → Definição.

HashMap

O Map mais usado. Sem ordem garantida.

- Acesso ultrarrápido por chave
- Ordem não garantida
- Permite chave e valor null
- Uso geral — o mais comum

TreeMap

Mantém as chaves em ordem natural.

- Chaves ordenadas automaticamente
- firstKey(), lastKey()
- Não permite chave null
- Ideal para relatórios ordenados

HashMap na prática:

```
import java.util.HashMap;  
import java.util.Map;  
  
public class ExemploMap {  
    public static void main(String[] args) {  
        // Chave = código do produto, Valor = nome  
        Map<Integer, String> estoque = new HashMap<>();  
  
        // Inserindo chave-valor  
        estoque.put(101, "Caneta Azul");  
        estoque.put(102, "Caderno A4");  
        estoque.put(103, "Borracha");  
  
        // Buscando pelo código  
        System.out.println(estoque.get(102)); // Caderno A4  
  
        // Verificando se chave existe  
        System.out.println(estoque.containsKey(999)); // false  
  
        // Percorrendo chaves e valores  
        for (Map.Entry<Integer, String> entry : estoque.entrySet()) {  
            System.out.println("Cód " + entry.getKey() + ": " + entry.getValue());  
        }  
  
        // Removendo  
        estoque.remove(101);  
        System.out.println("Tamanho: " + estoque.size()); // 2  
    }  
}
```

Métodos essenciais do Map:

Método	O que faz	Exemplo
<code>put(chave, valor)</code>	Insere ou atualiza	<code>map.put(1, "Ana")</code>
<code>get(chave)</code>	Retorna o valor	<code>map.get(1)</code>
<code>remove(chave)</code>	Remove a entrada	<code>map.remove(1)</code>
<code>containsKey(chave)</code>	Verifica se chave existe	<code>map.containsKey(5)</code>
<code>containsValue(val)</code>	Verifica se valor existe	<code>map.containsValue("Ana")</code>
<code>keySet()</code>	Conjunto de todas as chaves	<code>map.keySet()</code>
<code>values()</code>	Coleção de todos os valores	<code>map.values()</code>
<code>entrySet()</code>	Conjunto chave+valor	<code>map.entrySet()</code>
<code>size()</code>	Quantidade de pares	<code>map.size()</code>

Lista de Exercícios — Collections Framework

Coloque em prática List, Set e Map com exercícios do mundo real:

Exercício 1: Agenda Telefônica

Crie uma agenda usando HashMap onde a chave é o nome e o valor é o telefone. Implemente: adicionar contato, buscar por nome, listar todos e remover contato.

```
import java.util.*;

public class AgendaTelefonica {
    private Map<String, String> agenda = new HashMap<>();

    public void adicionar(String nome, String telefone) {
        agenda.put(nome, telefone);
        System.out.println("Contato " + nome + " adicionado!");
    }

    public void buscar(String nome) {
        String tel = agenda.get(nome);
        if (tel != null) {
            System.out.println(nome + ": " + tel);
        } else {
            System.out.println("Contato nao encontrado.");
        }
    }

    public void listarTodos() {
        if (agenda.isEmpty()) {
            System.out.println("Agenda vazia.");
            return;
        }
        // TreeMap para listar em ordem alfabética
    }
}
```

```

        new TreeMap<>(agenda).forEach((nome, tel) ->
            System.out.println(nome + " → " + tel)
        );
    }

    public void remover(String nome) {
        if (agenda.remove(nome) != null) {
            System.out.println(nome + " removido.");
        } else {
            System.out.println("Contato nao existe.");
        }
    }

    public static void main(String[] args) {
        AgendaTelefonica ag = new AgendaTelefonica();
        ag.adicionar("Ana", "(79) 99999-1111");
        ag.adicionar("Bruno", "(79) 98888-2222");
        ag.adicionar("Carlos", "(11) 97777-3333");
        ag.listarTodos();
        ag.buscar("Bruno");
        ag.remover("Ana");
        System.out.println("Total: " + ag.agenda.size());
    }
}

```

Exercício 2: Catálogo de Livros

Crie um catálogo usando ArrayList de objetos Livro (título, autor, ano). Implemente: adicionar livro, listar todos, buscar por autor e ordenar por título.

```

import java.util.*;

public class Livro {
    private String titulo, autor;
    private int ano;

    public Livro(String titulo, String autor, int ano) {
        this.titulo = titulo; this.autor = autor; this.ano = ano;
    }

    public String getTitulo() { return titulo; }
    public String getAutor() { return autor; }

    @Override
    public String toString() {
        return titulo + " - " + autor + " (" + ano + ")";
    }
}

public class Catalogo {
    private List<Livro> livros = new ArrayList<>();

    public void adicionar(Livro l) { livros.add(l); }

    public void listarTodos() {
        livros.forEach(System.out::println);
    }

    public void buscarPorAutor(String autor) {
        livros.stream()
    }
}

```

```

        .filter(l -> l.getAutor().equalsIgnoreCase(autor))
        .forEach(System.out::println);
    }

    public void ordenarPorTitulo() {
        livros.sort(Comparator.comparing(Livro::getTitulo));
    }

    public static void main(String[] args) {
        Catalogo cat = new Catalogo();
        cat.adicionar(new Livro("Clean Code", "Robert Martin", 2008));
        cat.adicionar(new Livro("Java Efetivo", "Joshua Bloch", 2018));
        cat.adicionar(new Livro("Design Patterns", "GoF", 1994));

        cat.ordenarPorTitulo();
        cat.listarTodos();
    }
}

```

Exercício 3: Controle de Produtos com Set e Map

Crie um controle de produtos usando HashMap (código → Produto). Use HashSet para armazenar categorias únicas. Mostre quantas categorias existem e liste produtos por categoria.

```

import java.util.*;

public class ControleProdutos {
    private Map<Integer, String[]> produtos = new HashMap<>();
    // String[] = {nome, categoria, preco}
    private Set<String> categorias = new HashSet<>();

    public void cadastrar(int codigo, String nome, String cat, double preco) {
        produtos.put(codigo, new String[]{nome, cat, String.valueOf(preco)});
        categorias.add(cat); // Set ignora duplicatas automaticamente
        System.out.println("Produto " + nome + " cadastrado.");
    }

    public void listarPorCategoria(String cat) {
        System.out.println("\n--- Categoria: " + cat + " ---");
        produtos.forEach((cod, dados) -> {
            if (dados[1].equalsIgnoreCase(cat)) {
                System.out.printf("Cód %d | %s | R$%s%n", cod, dados[0], dados[2]);
            }
        });
    }

    public void resumo() {
        System.out.println("\nTotal de produtos: " + produtos.size());
        System.out.println("Categorias (" + categorias.size() + "): " + categorias);
    }

    public static void main(String[] args) {
        ControleProdutos cp = new ControleProdutos();
        cp.cadastrar(1, "Caneta", "Papeleria", 2.50);
        cp.cadastrar(2, "Caderno", "Papeleria", 15.90);
        cp.cadastrar(3, "Mouse", "Informatica", 89.90);
        cp.cadastrar(4, "Teclado", "Informatica", 149.00);
        cp.cadastrar(5, "Borracha", "Papeleria", 1.20);
        cp.resumo();
        cp.listarPorCategoria("Papeleria");
    }
}

```



```
}  
}
```

Tratamento de Exceções

Erros acontecem — o usuário digita uma letra onde esperamos um número, o saldo não é suficiente, o arquivo não existe. O tratamento de exceções é a forma profissional de lidar com esses erros sem deixar o programa travar.

Analogia do Cinto de Segurança

Sem tratamento de exceções, um erro encerra o programa abruptamente — como um acidente sem cinto. Com tratamento de exceções, o programa "usa o cinto": detecta o problema, lida com ele de forma controlada, e continua funcionando.

6.4 Estrutura try / catch / finally

Bloco	Para que serve
try	Envolve o código que PODE lançar uma exceção
catch	Captura e trata a exceção se ela ocorrer
finally	Executa SEMPRE, com ou sem erro (limpeza, fechar recursos)
throw	Lança uma exceção manualmente
throws	Declara que um método pode lançar exceções

Exemplo básico:

```
public class ExemploTryCatch {  
    public static void main(String[] args) {  
        // Sem tratamento: programa TRAVA com ArithmeticException  
        // int resultado = 10 / 0; // ERRO!  
  
        // Com tratamento: programa CONTINUA  
        try {  
            int resultado = 10 / 0; // tenta executar  
            System.out.println("Resultado: " + resultado);  
        } catch (ArithmeticException e) {  
            System.out.println("Erro: divisao por zero!");  
            System.out.println("Detalhe: " + e.getMessage());  
        } finally {  
            System.out.println("Bloco finally - executa SEMPRE!");  
        }  
  
        System.out.println("Programa continua normalmente...");  
    }  
}
```

```
// Saída:
// Erro: divisao por zero!
// Detalhe: / by zero
// Bloco finally — executa SEMPRE!
// Programa continua normalmente...
```

6.5 Múltiplos catch — Capturando Erros Específicos

Você pode ter vários blocos catch para tratar diferentes tipos de erro. O Java percorre os catch de cima para baixo e executa o primeiro que corresponder ao erro.

```
import java.util.Scanner;

public class MultiCatch {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Digite um numero: ");
            int numero = Integer.parseInt(sc.nextLine()); // pode falhar

            int[] array = {10, 20, 30};
            System.out.println(array[numero]); // pode falhar

        } catch (NumberFormatException e) {
            System.out.println("Erro: voce nao digitou um numero!");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Erro: posicao fora do array!");
        } catch (Exception e) {
            // Captura qualquer outro erro nao previsto
            System.out.println("Erro inesperado: " + e.getMessage());
        }
    }
}
```

Exceções mais comuns no Java:

Exceção	Quando ocorre
NullPointerException	Tentar usar uma variável que é null
ArrayIndexOutOfBoundsException	Acessar índice inválido do array (ex: array[5] com 3 elementos)
NumberFormatException	Converter texto inválido para número: Integer.parseInt("abc")
ArithmeticException	Divisão por zero: 10 / 0
ClassCastException	Conversão inválida entre tipos incompatíveis
IllegalArgumentException	Argumento inválido passado para um método
StackOverflowError	Recursão infinita

6.6 throw e throws — Lançando Exceções

Além de capturar exceções, você pode lançar as suas próprias. Use `throw` para lançar e `throws` para avisar que um método pode lançar uma exceção.

```
// throws avisa que o método pode lançar uma exceção
public static void validarIdade(int idade) throws IllegalArgumentException {
    if (idade < 0 || idade > 150) {
        // throw lança a exceção
        throw new IllegalArgumentException("Idade invalida: " + idade);
    }
    System.out.println("Idade valida: " + idade);
}

public static void main(String[] args) {
    try {
        validarIdade(25);    // OK
        validarIdade(-5);   // LANÇA EXCEÇÃO
        validarIdade(200);  // nao chega aqui
    } catch (IllegalArgumentException e) {
        System.out.println("Erro: " + e.getMessage());
    }
}
```

6.7 Criando Suas Próprias Exceções

Você pode criar exceções personalizadas que fazem sentido para o seu sistema. Basta criar uma classe que herda de `Exception` (verificada) ou `RuntimeException` (não verificada).

```
// Exceção personalizada
public class SaldoInsuficienteException extends Exception {
    private double saldoAtual;
    private double valorSolicitado;

    public SaldoInsuficienteException(double saldo, double valor) {
        super("Saldo insuficiente! Disponível: R$" + saldo + " | Solicitado: R$" +
valor);
        this.saldoAtual = saldo;
        this.valorSolicitado = valor;
    }

    public double getSaldoAtual() { return saldoAtual; }
    public double getValorSolicitado() { return valorSolicitado; }
}

// Usando a exceção personalizada
public class ContaBancaria {
    private double saldo;

    public ContaBancaria(double saldoInicial) { this.saldo = saldoInicial; }

    public void sacar(double valor) throws SaldoInsuficienteException {
        if (valor > saldo) {
```

```

        throw new SaldoInsuficienteException(saldo, valor);
    }
    saldo -= valor;
    System.out.printf("Saque de R$%.2f realizado. Saldo: R$%.2f\n", valor,
saldo);
    }
}

```

PROJETO FINAL — Sistema Bancário com Validações

O que você vai construir:

Um sistema bancário completo que une Collections e Tratamento de Exceções. Este projeto simula funcionalidades reais de um banco.

- HashMap de contas bancárias (número → ContaBancaria)
- Exceções personalizadas: SaldoInsuficienteException, ContaNaoEncontradaException
- Menu interativo com validações em todas as operações
- Histórico de transações em ArrayList por conta

Exceções personalizadas:

```

// Exceção 1: saldo insuficiente para saque
public class SaldoInsuficienteException extends Exception {
    public SaldoInsuficienteException(double saldo, double valor) {
        super(String.format("Saldo insuficiente! Saldo: R$%.2f | Solicitado: R$%.2f",
saldo, valor));
    }
}

// Exceção 2: conta não encontrada
public class ContaNaoEncontradaException extends Exception {
    public ContaNaoEncontradaException(int numero) {
        super("Conta " + numero + " nao encontrada.");
    }
}

```

Classe ContaBancaria:

```

import java.util.ArrayList;
import java.util.List;

public class ContaBancaria {
    private int numero;
    private String titular;
    private double saldo;
    private List<String> historico = new ArrayList<>();
}

```

```

    public ContaBancaria(int numero, String titular, double saldoInicial) {
        this.numero = numero;
        this.titular = titular;
        this.saldo = saldoInicial;
        historico.add("Conta aberta com R$" + saldoInicial);
    }

    public void depositar(double valor) throws IllegalArgumentException {
        if (valor <= 0) throw new IllegalArgumentException("Valor deve ser positivo!");
        saldo += valor;
        historico.add("Deposito: +R$" + valor + " | Saldo: R$" + saldo);
        System.out.printf("Deposito de R$%.2f realizado!\n", valor);
    }

    public void sacar(double valor) throws SaldoInsuficienteException,
        IllegalArgumentException {
        if (valor <= 0) throw new IllegalArgumentException("Valor deve ser positivo!");
        if (valor > saldo) throw new SaldoInsuficienteException(saldo, valor);
        saldo -= valor;
        historico.add("Saque: -R$" + valor + " | Saldo: R$" + saldo);
        System.out.printf("Saque de R$%.2f realizado!\n", valor);
    }

    public void exibirHistorico() {
        System.out.println("\n=== Conta " + numero + " | " + titular + " ===");
        System.out.printf("Saldo atual: R$%.2f\n", saldo);
        System.out.println("--- Historico ---");
        historico.forEach(System.out::println);
    }

    public int getNumero() { return numero; }
    public String getTitular() { return titular; }
    public double getSaldo() { return saldo; }
}

```

Sistema principal com menu:

```

import java.util.*;

public class SistemaBancario {
    private Map<Integer, ContaBancaria> contas = new HashMap<>();
    private int proximoNumero = 1001;
    private Scanner sc = new Scanner(System.in);

    public ContaBancaria buscarConta(int numero) throws ContaNaoEncontradaException {
        ContaBancaria conta = contas.get(numero);
        if (conta == null) throw new ContaNaoEncontradaException(numero);
        return conta;
    }

    public void abrirConta() {
        try {
            System.out.print("Nome do titular: ");
            String nome = sc.nextLine();
            System.out.print("Deposito inicial: R$ ");
            double inicial = Double.parseDouble(sc.nextLine());

            ContaBancaria nova = new ContaBancaria(proximoNumero, nome, inicial);
            contas.put(proximoNumero, nova);
            System.out.println("Conta " + proximoNumero + " aberta!");
        }
    }
}

```

```

        proximoNumero++;
    } catch (NumberFormatException e) {
        System.out.println("Valor invalido! Digite apenas numeros.");
    }
}

public void realizarOperacao(String tipo) {
    try {
        System.out.print("Numero da conta: ");
        int num = Integer.parseInt(sc.nextLine());
        System.out.print("Valor: R$ ");
        double val = Double.parseDouble(sc.nextLine());

        ContaBancaria conta = buscarConta(num);
        if (tipo.equals("deposito")) conta.depositar(val);
        else conta.sacar(val);

    } catch (NumberFormatException e) {
        System.out.println("Entrada invalida!");
    } catch (ContaNaoEncontradaException e) {
        System.out.println("Erro: " + e.getMessage());
    } catch (SaldoInsuficienteException e) {
        System.out.println("Erro: " + e.getMessage());
    } catch (IllegalArgumentException e) {
        System.out.println("Erro: " + e.getMessage());
    }
}

public void menu() {
    int opcao;
    do {
        System.out.println("\n=== BANCO JAVA ===");
        System.out.println("1 - Abrir conta    2 - Depositar");
        System.out.println("3 - Sacar        4 - Extrato");
        System.out.println("5 - Listar contas 6 - Sair");
        System.out.print("Opcao: ");

        try { opcao = Integer.parseInt(sc.nextLine()); }
        catch (NumberFormatException e) { opcao = 0; }

        switch (opcao) {
            case 1: abrirConta(); break;
            case 2: realizarOperacao("deposito"); break;
            case 3: realizarOperacao("saque"); break;
            case 4:
                try {
                    System.out.print("Numero da conta: ");
                    int n = Integer.parseInt(sc.nextLine());
                    buscarConta(n).exibirHistorico();
                } catch (Exception e) {
                    System.out.println(e.getMessage());
                }
                break;
            case 5:
                contas.values().forEach(c ->
                    System.out.printf("Conta %d | %s | R$%.2f%n",
                        c.getNumero(), c.getTitular(), c.getSaldo()));
                break;
            case 6: System.out.println("Encerrando..."); break;
            default: System.out.println("Opcao invalida!");
        }
    } while (opcao != 6);
}

```

```
public static void main(String[] args) {  
    new SistemaBancario().menu();  
}
```

Resumo dos Conceitos

Conceito	Interface/Classe	Quando usar
<code>ArrayList</code>	<code>List</code>	Lista ordenada com acesso por índice
<code>LinkedList</code>	<code>List</code> / <code>Queue</code>	Muitas inserções/remoções no meio
<code>HashSet</code>	<code>Set</code>	Garantir elementos únicos sem ordem
<code>TreeSet</code>	<code>Set</code>	Elementos únicos e sempre ordenados
<code>HashMap</code>	<code>Map</code>	Busca ultrarrápida por chave
<code>TreeMap</code>	<code>Map</code>	Pares chave-valor sempre ordenados pela chave
<code>try/catch</code>	—	Capturar e tratar erros
<code>finally</code>	—	Executar sempre (fechar recursos, logs)
<code>throw</code>	—	Lançar exceção manualmente
<code>Exception</code> customizada	<code>extends Exception</code>	Erros específicos do seu domínio

Você dominou Collections e Exceções, que são usados em 100% dos projetos Java reais. Agora você está pronto para qualquer desafio!